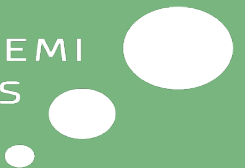


Barrierebetingelser



Hvad er barrierebetingelser?

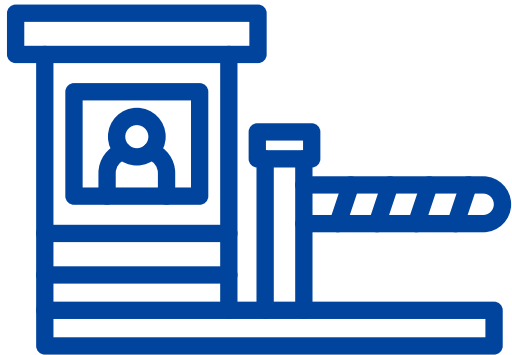
Barrierebetingelser er standarder eller **kontrolpunkter** inden for udviklingslivscyklussen, der bruges til at måle succes eller fiasko.

Formål - Sikre, at produktet stemmer overens med sin vision og sine mål, forbedre kvalitet og reducere udviklingsomkostninger.

Anvendelse - Kan implementeres i enhver udviklingsmetode—**Agile**, **Vandfald** eller **Hybrid**.

Eksempler

- Arkitektur Review Board (ARB) sessioner
- Kode-review
- Performancetest
- Udførelse af *unit tests* før udgivelse



Barrierebetingelser i Agile Udvikling

Almindelig Misforståelse:

At inkorporere standarder og processer **krænker** den *Agile* tankegangen.

Afklaring:

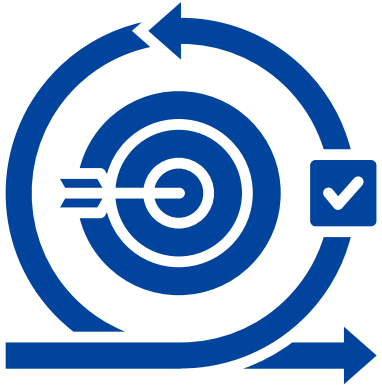
- **Agile** værdsætter individer og interaktioner **over** processer, men afviser ikke processer helt.
- Processer tilføjer værdi, når de balanceres korrekt med menneskelige interaktioner.

Integration med Agile:

- Barrierebetingelser kan inkluderes uden overdreven dokumentation.
- ARB og Joint Architecture Design (JAD) sessioner sikrer skalerbarhed og kvalitet.

Fordele:

- Understøtter fungerende software ved at fange problemer tidligt.
- Fremmer kundesamarbejde ved at tilpasse produkter til kundens behov.
- Hjælper teams med effektivt at reagere på ændringer.



Barriererbetingelser i Vandfaldsudvikling

Traditionel Implementering:

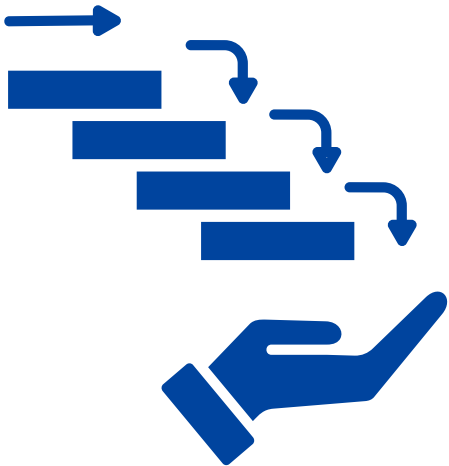
Barrierebetingelser fungerer som indgangs- og udgangskriterier for hver fase.

Eksempler:

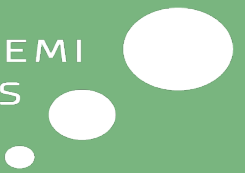
- **Kravfasen:** Udgangskriterier: Godkendelser fra interessenter, kravgennemgange.
- **Designfasen:** ARB-sessioner som udgangskriterier for at sikre overensstemmelse med arkitektoniske principper.
- **Kodefasen:** Kodegennemgang for konsistens med designet.
- **Testfasen:** Performancetest før validering.
- **Vedligeholdelsesfasen:** Implementering af produktionsmålinger.

Formål:

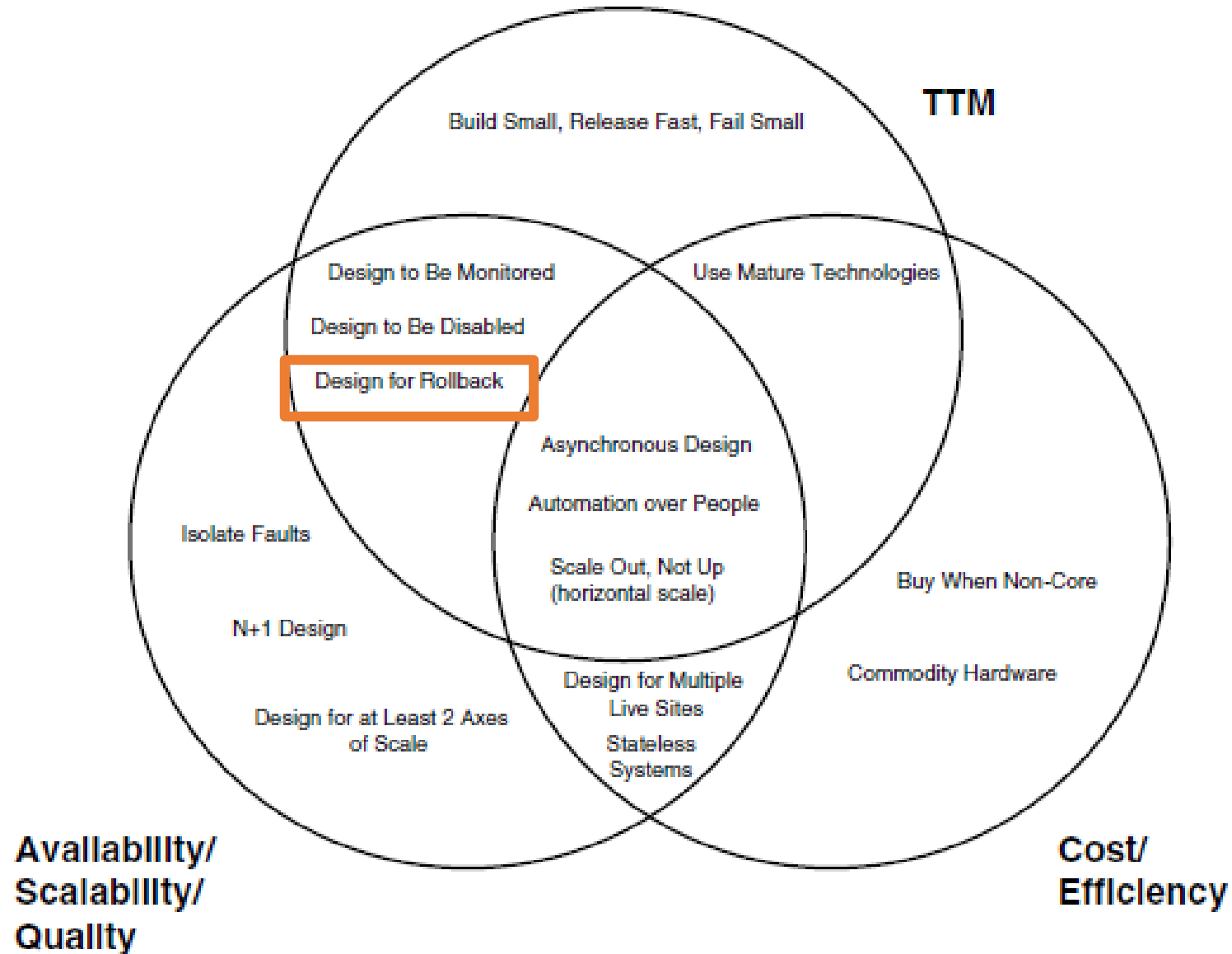
Fange problemer tidligt for at minimere genarbejde og sikre kvalitet.



Rollback



Arkitektur Principper i Operations

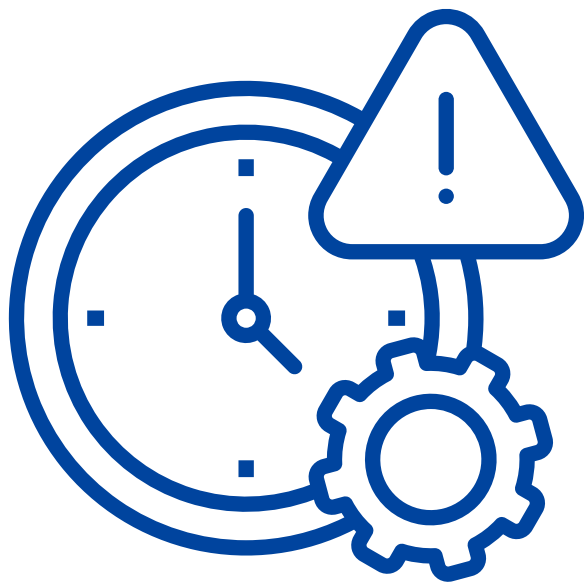


Hvorfor Rollback?

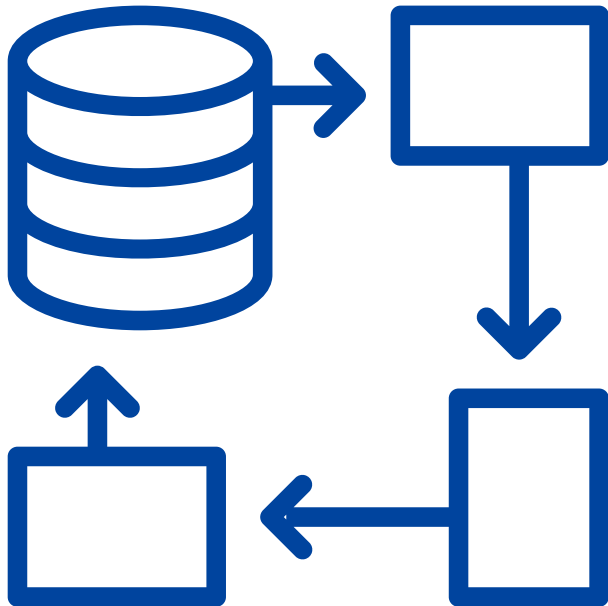


- Rollback-procedureen er nødvendig trods effektive barrierer
- Tjener som en **forsikring** mod uforudsete problemer
- Minimerer negativ påvirkning på kunder ved hurtig genopretning

Rollback-vinduet (hvornår?)



- Rollback-vinduets varighed varierer efter virksomhedens behov
- Kræver tilstrækkelig dataindsamling for at vurdere performance
- Nye funktioner kan have længere adoptionskurver og kræver længere observationsperiode

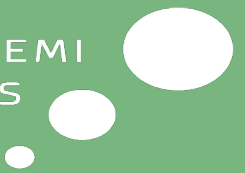


- Database-schema inkompatibilitet som udfordring
- Løsninger: gradvis skemaudvidelse og bevarelse af gamle datarelationer
- Brug af dataoverførselsværktøjer som triggers, *daemons* eller *replikationsteknologier*

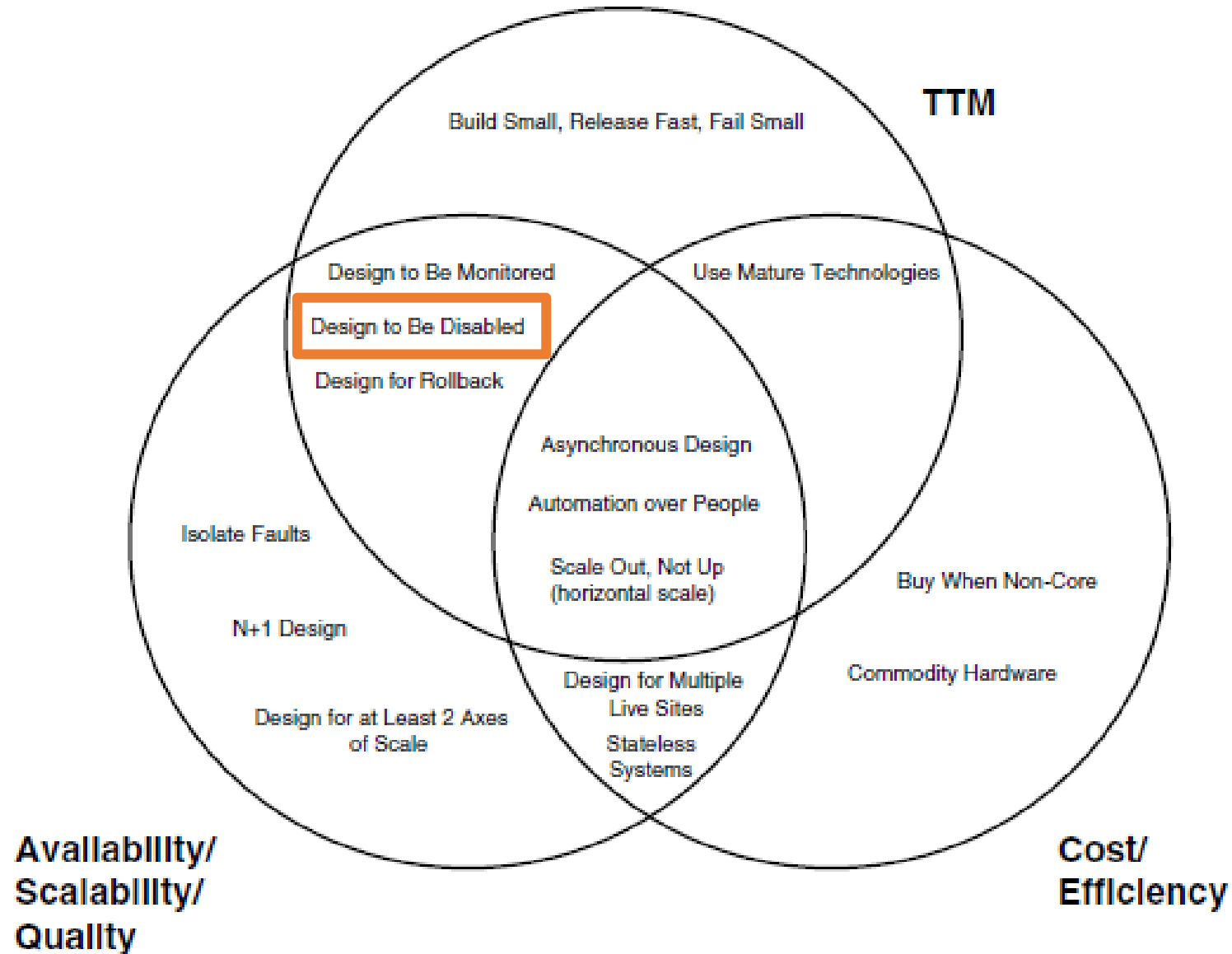


- Implementering kan øge **release-omkostningerne** med op til 20%
- Ledelsen bør godkende risikoen ved **fravigelse** af rollback-princippet
- Store, komplekse releases har størst behov for rollback-muligheder

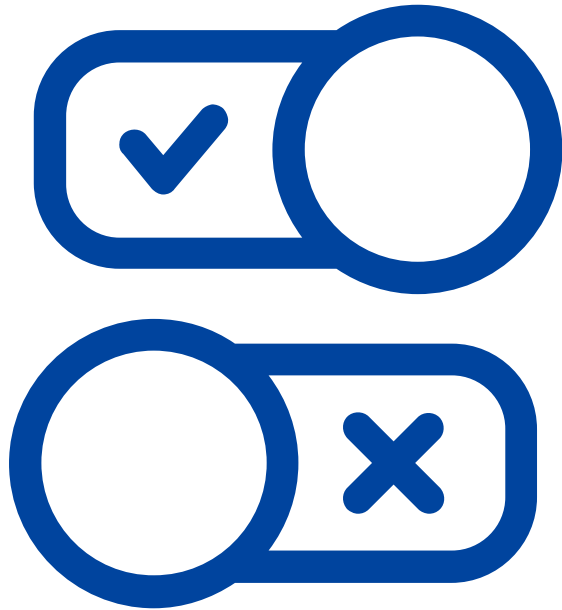
Markdown Functionality



Arkitektur Principper i Operations



Markdown: Design til at Blive Deaktiveret



- **Hurtigere** at deaktivere en funktion end at udføre en fuld rollback
- **Tillader andre nye** og modificerede funktioner at fortsætte med at fungere
- **Omkostningstungt**, men øger fleksibiliteten og understøtter agile metoder

Opgave M8.02

Canvas opgaver

Case: Rollback vs. Markdown



Gruppe